МИРЭА — Российский технологический университет
Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 7

Контейнеризация. Docker и Docker Compose

по дисциплине «Инструменты девопс»

Выполнил студент группы ЭФБО-18-24

Кудояров Денис Рустемович

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Рабочее окружение 4
2 Выполнение практической работы 5
2.1 Подготовка Docker Engine 5
2.2 Dockerfile и первый запуск без базы 6
2.3 Изменение приложения и кэш слоёв 8
2.4 Compose, готовность базы и сеть 9
2.5 Сохранность данных 11
2.6 Обход UFW при публикации Docker 12
2.7 Обратный прокси и диагностика 502 13
2.8 Аудит, документация и контрольная точка 15
2.9 Контрольные точки VirtualBox 16
3 Ответы на контрольные вопросы 18
3.1 Изоляция контейнеров 18
3.2 Кэш слоёв 18
3.3 localhost внутри контейнера 18
3.4 Проверка готовности 18
3.5 Жизненный цикл данных 18
3.6 Docker и UFW 19
3.7 Полномочия группы docker 19
3.8 Диагностика 502 19
ЗАКЛЮЧЕНИЕ 20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 21
ПРИЛОЖЕНИЕ А Dockerfile 22
ПРИЛОЖЕНИЕ Б Docker Compose 23
ПРИЛОЖЕНИЕ В История собранного образа 24
ПРИЛОЖЕНИЕ Г Раздел 9 инструкции восстановления 26

ВВЕДЕНИЕ

Практическая работа выполнена по методическим указаниям [1]. Исходные тексты и материалы проверок размещены в репозитории [2], итоговые изменения представлены в Pull Request [3].

Цель — упаковать Flask-приложение в контейнер, подключить PostgreSQL с постоянным хранением данных и опубликовать приложение через обратный прокси Nginx. Изучены слои и кэш образов, права процесса, сеть Compose, готовность базы и особенности фильтрации портов Docker.

Выполнены аудиторная и самостоятельная части. Использована виртуальная машина предыдущих работ; статический ресурс Nginx сохранён.

При оформлении применены правила ГОСТ 7.32–2017 для текста, нумерации, иллюстраций, таблиц и приложений [4] с сохранением структуры учебной практической работы.

При настройке и проверке программных средств использована документация, приведённая в источниках [5–10].

1 Рабочее окружение

Данные приведены в таблице 1.

Таблица 1 — Параметры рабочего окружения

Параметр	Значение
ОС компьютера	Windows 11
Гипервизор	Oracle VirtualBox 7.2.20
ОС гостя	Ubuntu Server 24.04.5 LTS amd64
Ресурсы	3072 МБ ОЗУ, 1 vCPU, динамический диск 25,1 ГиБ
Docker Engine	Community 29.9.2, официальный APT-репозиторий Docker
Docker Compose	Плагин v5.6.1
Приложение	Flask 3.0.3, Gunicorn 22.0.0, psycopg2-binary 2.9.9
Образы	python:3.12-slim; postgres:16-alpine; devops-app:1.0
Проект	/home/devops/app; сеть app_backend; том app_db_data
Публикация	https://devops.local/app/ → 127.0.0.1:5000

2 Выполнение практической работы

2.1 Подготовка Docker Engine

Docker Engine установлен внутри Ubuntu из официального APT-репозитория Docker для noble/stable. Проверены разделы Client и Server, версия Compose и запуск hello-world. devops добавлен в группу docker; доступ без sudo проверен в новой SSH-сессии. Docker Desktop на компьютере не используется как замена демону внутри VM.

Перед контейнеризацией сохранён снимок 04-nginx-https. Виртуальная машина имеет 1 vCPU и 3072 МБ памяти из-за выявленной нестабильности VirtualBox через Hyper-V/NEM при нескольких виртуальных процессорах.

Docker работает от учебного пользователя

```
uid=1001(devops) gid=1001(devops)
groups=1001(devops),27(sudo),988(docker)
Client: Docker Engine - Community
 Version:           29.9.2
 API version:       1.56
 Go version:        go1.26.10
 Git commit:        a82e528
 Built:             Tue Oct 20 18:44:40 2026
 OS/Arch:           linux/amd64
 Context:           default
Server: Docker Engine - Community
 Engine:
  Version:          29.9.2
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.10
  Git commit:       0367cbe
  Built:            Tue Oct 20 18:44:40 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.4.2
```

```
 GitCommit:        b7a25261e029d68978bbf290df0a62fc819b3c51
runc:
 Version:          1.5.2
 GitCommit:        v1.5.2-0-g29dd3dc2
docker-init:
 Version:          0.19.0
 GitCommit:        de40ad0
Docker Compose version v5.6.1
Hello from Docker!
This message shows that your installation appears to be working
correctly.
To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker
Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which
runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which
sent it
    to your terminal.
To try something more ambitious, you can run an Ubuntu container with:
```

2.2 Dockerfile и первый запуск без базы

Созданы Dockerfile, requirements.txt, app.py и .dockerignore. Образ основан на python:3.12-slim. Зависимости устанавливаются отдельным слоем до копирования приложения. Последние инструкции USER app и exec-форма CMD запускают Gunicorn от непривилегированного пользователя; EXPOSE описывает порт, но сам не публикует его.

Файлы доставлены rsync: сначала --dry-run, затем фактическая синхронизация. Во всех доставках исключён .env, чтобы --delete не удалял серверный секрет. Контекст сборки тоже исключает .env через .dockerignore.

В первом образе сохранено исходное приветствие Hello from Docker!. Контейнер app-test запущен с публикацией только на loopback. /health вернул 200 OK, а / — 500 Internal Server Error. В журнале записан psycopg2.OperationalError: Connection refused для localhost:5432. DB_HOST ещё не задан, поэтому приложение пытается соединиться с самим собой; localhost контейнера не является loopback сервера или соседнего контейнера.

Внутри контейнера id показал uid=999(app). Файл /app/app.py имеет владельца root и права 644: процесс читает код и не может его менять. После опыта app-test удалён. Полный вывод docker history сохранён в приложении В.

Запуск без СУБД и идентификатор пользователя

```
dff971d40106219f173043c18528eaff61cd68ca85ecd81d11ad99cf714d9a45
CONTAINER ID   IMAGE            COMMAND                  CREATED
STATUS         PORTS                       NAMES
dff971d40106   devops-app:1.0   "gunicorn --bind 0.0…"   4 seconds ago
Up 3 seconds   127.0.0.1:5000->5000/tcp   app-test
HTTP/1.1 200 OK
Server: gunicorn
Date: Thu, 22 Oct 2026 21:33:45 GMT
Connection: close
Content-Type: text/html; charset=utf-8
Content-Length: 3
ok
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: gunicorn
Date: Thu, 22 Oct 2026 21:33:45 GMT
Connection: close
Content-Type: text/html; charset=utf-8
Content-Length: 265
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
```

```
<p>The server encountered an internal error and was unable to complete
your request. Either the server is overloaded or there is an error in
the application.</p>
172.17.0.1 - - [22/Oct/2026:21:33:45 +0000] "GET /health HTTP/1.1" 200
3 "-" "curl/8.5.0"
172.17.0.1 - - [22/Oct/2026:21:33:45 +0000] "GET / HTTP/1.1" 500 265
"-" "curl/8.5.0"
uid=999(app) gid=999(app) groups=999(app)
644 root:root /app/app.py
app-test
STDERR
```

Причина ошибки соединения

```
 connection to server at "localhost" (::1), port 5432 failed:
Connection refused
    Is the server running on that host and accepting TCP/IP
connections?
connection to server at "localhost" (127.0.0.1), port 5432 failed:
Connection refused
    Is the server running on that host and accepting TCP/IP
connections?
```

2.3 Изменение приложения и кэш слоёв

Приветствие заменено на Hello from Docker, Kudoyarov!. После повторной доставки слой COPY app.py пересобран за 0,2 с. Создание пользователя, WORKDIR, COPY requirements.txt и RUN pip install отмечены CACHED: их входные данные не изменились. Поэтому зависимости повторно не скачивались.

Если сначала выполнить COPY . ., любое изменение app.py изменит родительский слой для pip install и потребует заново установить зависимости. Раздельное копирование сокращает объём повторной работы.

Повторная сборка после изменения только app.py

```
 RUN useradd --system --no-create-home app
#6 CACHED
```

```
#7 [3/6] WORKDIR /app
#7 CACHED
#8 [4/6] COPY requirements.txt .
#8 CACHED
#9 [5/6] RUN pip install --no-cache-dir -r requirements.txt
#9 CACHED
#10 [6/6] COPY app.py .
#10 DONE 0.2s
#11 exporting to image
#11 exporting layers
#11 exporting layers 0.2s done
#11 exporting manifest
sha256:8196b92beee0185ab7b41fbec7a22269362170d7ebc5aed99f68a6082c8cb69
5
#11 exporting manifest
sha256:8196b92beee0185ab7b41fbec7a22269362170d7ebc5aed99f68a6082c8cb69
5 0.0s done
#11 exporting config
sha256:33f70355acacdca4f874aaf73d8375f9c948fb3a78e228bb3243473de48f5e1
5 0.0s done
#11 exporting attestation manifest
sha256:8d68cc04409b0466d2558dc7083a6168c75e1849849b14b3147e215794181ec
b
```

2.4 Compose, готовность базы и сеть

В docker-compose.yml описаны db и app в пользовательской сети backend. PostgreSQL 16-alpine хранит данные в именованном томе db_data. Healthcheck выполняет pg_isready; depends_on с service_healthy ждёт готовности базы перед стартом приложения. Двойной знак $$ в healthcheck оставляет подстановку переменных оболочке контейнера.

Шаблон .env.example содержит заглушку. Реальный .env создан только в /home/devops/app со случайным паролем openssl rand -hex 16, владельцем

devops и правами 600. Он отсутствует в Git и в собранном образе. Содержимое секретного файла в отчёт не выводится.

Имя сервиса db разрешается встроенным DNS Docker в 172.18.0.2. Приложение использует DB_HOST=db, а не адрес localhost. На хосте VM опубликован только 127.0.0.1:5000; база имеет внутренний 5432/tcp без привязки к порту хоста. Три последовательных обращения дали Visits: 3, 4, 5.

При автоматизации docker compose exec -T сначала потреблял стандартный ввод сценария. Для проверок явно задано --interactive=false, после чего остальные команды выполняются последовательно. В Compose v5.6.1 команда port db 5432 выводила :0 для отсутствующей публикации; реальное отсутствие привязки проверено через docker inspect и ss.

Три обращения и проверка сети

```
, Kudoyarov! Visits: 3
Hello from Docker, Kudoyarov! Visits: 4
Hello from Docker, Kudoyarov! Visits: 5
NETWORK ID     NAME          DRIVER    SCOPE
306290f5e1f5   app_backend   bridge    local
6b2177ffa958   bridge        bridge    local
d15a6fd868b4   host          host      local
3257067ed4d2   none          null      local
172.18.0.2      db
END
```

Результат проверки представлен на рисунке 1.



Рисунок 1 — Состояние Compose, пользователь, счётчик, сеть и права .env

2.5 Сохранность данных

Проведены два опыта только с учебным проектом app и его томом app_db_data. Перед down счётчик равен 1; после down и повторного up получено 2. Контейнеры и сеть были пересозданы, а именованный том остался, поэтому предыдущая запись не исчезла.

После down -v том удалён. Новый up создал пустую базу, и первый запрос дал 1. Это подтверждает, что постоянные данные расположены в томе, а не в изменяемом слое контейнера. После опыта проект оставлен работающим; действующая база и том сохранены для следующей работы.

Данные приведены в таблице 2.

Таблица 2 — Сохранность данных

Момент	Visits	Состояние данных
Перед down	1	В базе одна запись
После down → up	2	Старая запись сохранена, добавлена новая
После down -v → up	1	Создана новая пустая база

Фактический результат опыта с томом

```
BEFORE_DOWN
```

```
Hello from Docker, Kudoyarov! Visits: 1
DRIVER    VOLUME NAME
local     app_db_data
AFTER_DOWN
Hello from Docker, Kudoyarov! Visits: 2
AFTER_DOWN_V
Hello from Docker, Kudoyarov! Visits: 1
DRIVER    VOLUME NAME
local     app_db_data
STDERR
 Container app-app-1 Stopping
```

2.6 Обход UFW при публикации Docker

Временно запущен отдельный контейнер exposed с -p 5001:5000. Разрешения 5001 в UFW нет, однако в таблице NAT Docker создано перенаправление и с Windows адрес http://devops.local:5001/health вернул 200 OK и ok. Контейнер сразу остановлен и удалён.

Трафик опубликованных портов проходит через цепочки перенаправления Docker и не обрабатывается так же, как обычный локальный входящий сокет UFW. Поэтому публикация на всех адресах способна открыть порт несмотря на deny incoming. В постоянной конфигурации используется 127.0.0.1:5000:5000; доступ извне идёт только через Nginx.

Правило Docker для временного порта

```
da7e4c8d813d4acbc40cf711b2b25d17b953d01435f674393bb426d0d895b34a
DNAT       6    --  0.0.0.0/0            0.0.0.0/0            tcp
dpt:5001 to:172.17.0.2:5000
EXIT STATUS: 0
```

Ответ на запрос с Windows

```
HTTP/1.1 200 OK
Server: gunicorn
Date: Thu, 22 Oct 2026 21:36:43 GMT
Connection: close
Content-Type: text/html; charset=utf-8
Content-Length: 3
ok
```

2.7 Обратный прокси и диагностика 502

В HTTPS-блок Nginx добавлен location /app/ с proxy_pass http://127.0.0.1:5000/. Завершающий слеш заменяет префикс /app/: /app/health поступает приложению как /health. Передаются Host, X-Real-IP, X-Forwarded-For и X-Forwarded-Proto. Перед reload успешно выполнен nginx -t.

Проверены главная страница приложения, health и статический ресурс: приложение отвечает персональным приветствием и счётчиком, /app/health — ok, статический / — HTTP 200. На Windows использован сертификат devops.crt с --cacert. Для диагностических обращений из VM добавлен локальный псевдоним devops.local к 127.0.0.1 в /etc/hosts.

При остановке только app Nginx продолжил принимать HTTPS, но вернул 502. Журнал содержит connect() failed (111: Connection refused) while connecting to upstream и адрес http://127.0.0.1:5000/. Отказ произошёл на участке Nginx → приложение. После docker compose start app ответ /app/health снова равен 200.

Ответ при остановленном приложении

```
HTTP/1.1 502 Bad Gateway
Server: nginx/1.24.0 (Ubuntu)
Date: Thu, 22 Oct 2026 21:36:45 GMT
Content-Type: text/html
Content-Length: 166
```

Запись об отказе upstream

```
[error] 4848#4848: *7 connect() failed (111: Connection refused)
while connecting to upstream, client: 127.0.0.1, server: devops.local,
request: "HEAD /app/ HTTP/1.1", upstream: "http://127.0.0.1:5000/",
host: "devops.local"
```

Результат проверки представлен на рисунке 2.



Рисунок 2 — Приложение, опубликованное через HTTPS

Результат проверки представлен на рисунке 3.

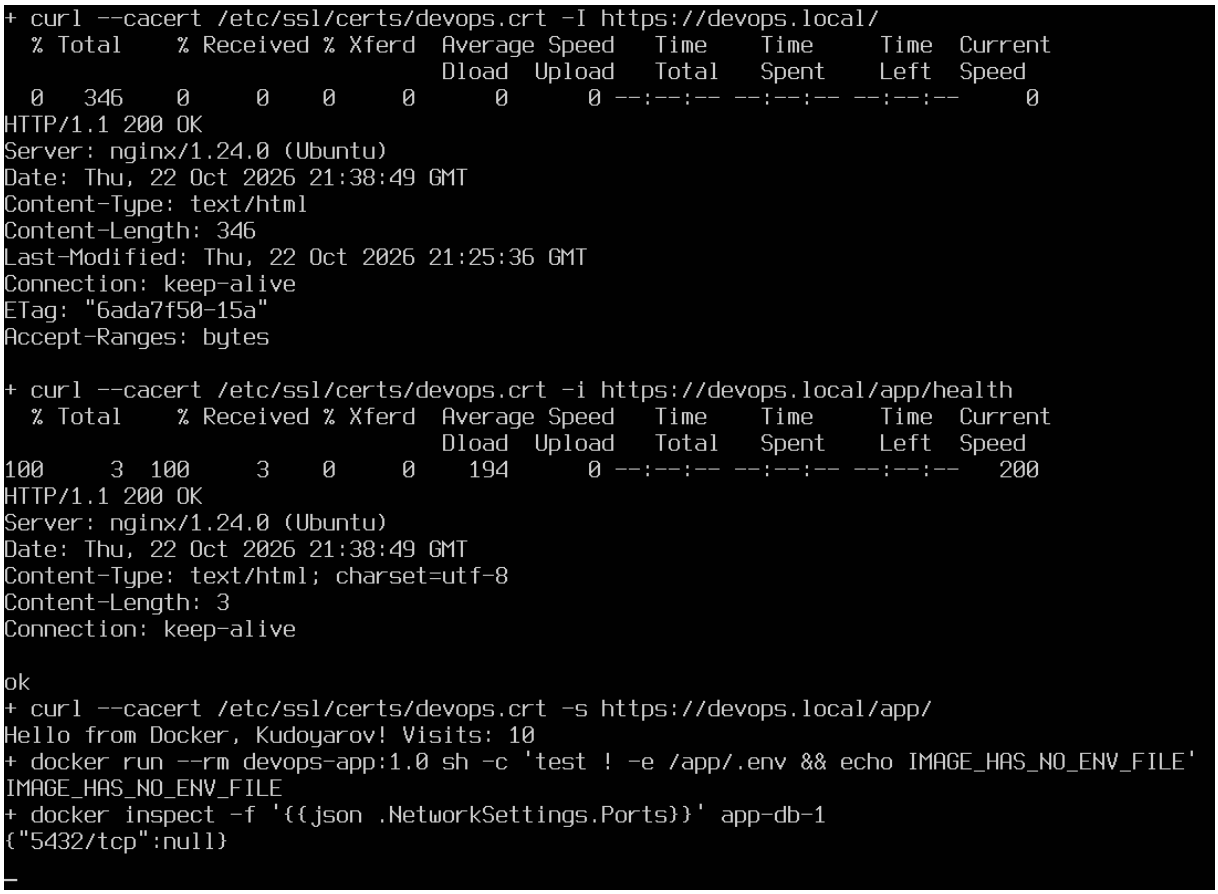


Рисунок 3 — Прокси, сохранённый статический ресурс и отсутствие .env в образе

2.8 Аудит, документация и контрольная точка

Раздел [5] сценария audit.sh проверяет службу Docker, healthy базы, безопасную публикацию приложения и отсутствие публикации СУБД, режим .env 600. При штатной настройке пройдены все 14 проверок, код 0.

Для отрицательной проверки 127.0.0.1:5000:5000 временно заменено на 5000:5000, проект пересоздан. ss показал внешние слушатели, проверка портов завершилась ошибкой, итоговый код 1. После возврата исходного Compose снова получено 14 успешных проверок и 0. Временный небезопасный вариант не оставлен в Git.

docs/server-setup.md дополнен разделом 9 с версиями, способом установки, группой docker, путями, правами, сетью, томом и полным фрагментом прокси. Создан снимок 05-docker-compose штатно выключенной VM; после запуска повторно проверены сервисы. Исходные тексты и отчёты оформлены в отдельной ветке Tools_DevOps; состояние Pull Request приведено в источниках.

Намеренно нарушенная публикация порта

```
[OK] Docker active
[OK] Database healthy
[FAIL] App bound to loopback and database unpublished expected=yes
actual=no
[OK] Environment file mode
Passed: 13; failed: 1
AUDIT_EXIT=1
STDERR
 Container app-db-1 Running
 Container app-app-1 Recreate
```

Результат проверки представлен на рисунке 4.



Рисунок 4 — Успешный аудит SSH, UFW, Nginx и контейнеров

2.9 Контрольные точки VirtualBox

Ниже приведён фактический список контрольных точек, полученный VBoxManage. Дополнительные исходные и восстановительные снимки оставлены для сохранения истории.

Данные приведены в таблице 3.

Таблица 3 — Контрольные точки виртуальной машины

Снимок	UUID
00-before-practices-5-8	16f97eae-c664-4595-ad24-43eed47f8150
00b-installed-before-first-boot	bfa99cdd-8157-464b-88de-2dedfb16c8f1
01-clean-install	c9fdcdf2-3b52-402a-bdaa-06bd85b8fed8
00c-recovery-after-live-snapshot	bd7e0d59-77ea-42c1-a29c-353fafc2d17b
02-keys-configured	b12d6c00-f179-420b-9f32-34229bcc8ad8
03-ssh-hardened	c7d97ff6-549d-4b1a-8ea1-42489e7236f1
04-nginx-https	8901ccc1-bcf7-4284-af3a-d8894d24f04b
05-docker-compose	773c7579-32f6-40d3-b51a-228e4040483f

3 Ответы на контрольные вопросы

3.1 Изоляция контейнеров

Пространства имён ядра изолируют процессы, сеть и другие ресурсы, а cgroups учитывают и ограничивают потребление ресурсов. Контейнер использует ядро хоста и запускает процесс без загрузки отдельной операционной системы, поэтому стартует быстрее виртуальной машины.

3.2 Кэш слоёв

Зависимости копируются и устанавливаются до копирования app.py. Изменение только приложения оставляет слой установки зависимостей пригодным для повторного использования. COPY . . перед pip install связал бы установку зависимостей с любым изменением файлов в контексте.

3.3 localhost внутри контейнера

localhost указывает на сам контейнер, а не на сервер или контейнер базы. В пользовательской сети Compose приложение обращается к имени сервиса db, которое встроенный DNS Docker преобразует в адрес контейнера PostgreSQL.

3.4 Проверка готовности

Обычный depends_on задаёт порядок запуска, но не готовность базы принимать соединения. Условие service_healthy ожидает успешного pg_isready и предотвращает старт приложения до готовности PostgreSQL.

3.5 Жизненный цикл данных

compose down удаляет контейнеры и сеть, сохраняя именованный том. down -v удаляет также том и данные. Тег postgres:16-alpine удерживает основную версию, тогда как latest может перейти на несовместимый формат данных другой основной версии.

3.6 Docker и UFW

Docker добавляет правила перенаправления трафика в свои цепочки, поэтому публикация на всех интерфейсах может открыть порт вопреки ожидаемой политике UFW. Для приложения за локальным Nginx используется 127.0.0.1:5000:5000, а порт базы не публикуется.

3.7 Полномочия группы docker

Участник группы управляет системным демоном Docker и может запустить контейнер с каталогом / сервера, смонтированным на запись. Это позволяет изменять файлы системы с полномочиями root, поэтому членство фактически предоставляет административный доступ. Демонстрационная команда docker run --rm -it -v /:/host alpine chroot /host /bin/sh запускает оболочку с корнем файловой системы сервера. В данной работе такой доступ не использовался: пример поясняет последствия членства в группе.

3.8 Диагностика 502

Возможные причины — остановленное приложение, неверный порт или адрес upstream. Сначала проверяют код ответа и error.log Nginx, затем docker compose ps и logs, опубликованный порт и прямой запрос с сервера к 127.0.0.1:5000. Ошибка соединения с upstream указывает на участок Nginx — приложение.

ЗАКЛЮЧЕНИЕ

Собран образ непривилегированного Flask-приложения и проверено повторное использование слоёв. Compose запускает приложение после готовности PostgreSQL, обеспечивает разрешение имени db и хранение данных в именованном томе.

Приложение опубликовано через Nginx по HTTPS, прямой внешний доступ к 5000 и 5432 закрыт конфигурацией публикации. Практически воспроизведены обход UFW временным контейнером и отказ upstream с кодом 502. После опытов восстановлена штатная конфигурация; итоговый аудит возвращает 0.

При работе учтены особенности Compose v5.6.1: отсутствие публикации базы проверяется фактическими привязками, а не строкой :0; неинтерактивные exec-команды запускаются с --interactive=false. Сохранены конфигурация, том и снимок для последующих занятий.

Полные тексты сценариев, конфигураций и дополнительные материалы приведены в приложениях А, Б, В, Г.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Практическая работа № 7 : методические указания по дисциплине «Инструменты девопс» / Д. М. Гиматдинов. — 2026. — 14 с. — Текст : электронный.

2. Tools_DevOps : учебный репозиторий. — URL: https://github.com/Hehekutov/Tools_DevOps (дата обращения: 23.10.2026). — Текст : электронный.

3. Практические работы № 5–7 : Pull Request № 3. — URL: https://github.com/Hehekutov/Tools_DevOps/pull/3 (дата обращения: 23.10.2026). — Текст : электронный.

4. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — URL: https://protect.gost.ru/gost/details/7d280e43-7036-4a69-8e6e-d15867028343 (дата обращения: 23.10.2026). — Текст : электронный.

5. Установка Docker Engine. — URL: https://docs.docker.com/engine/install/ubuntu/ (дата обращения: 23.10.2026). — Текст : электронный.

6. Dockerfile. — URL: https://docs.docker.com/reference/dockerfile/ (дата обращения: 23.10.2026). — Текст : электронный.

7. Кэш сборки. — URL: https://docs.docker.com/build/cache/ (дата обращения: 23.10.2026). — Текст : электронный.

8. Compose. — URL: https://docs.docker.com/reference/compose-file/ (дата обращения: 23.10.2026). — Текст : электронный.

9. Сеть и UFW. — URL: https://docs.docker.com/engine/network/packet-filtering-firewalls/ (дата обращения: 23.10.2026). — Текст : электронный.

10. Nginx proxy. — URL: https://nginx.org/ru/docs/http/ngx_http_proxy_module.html (дата обращения: 23.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А

Dockerfile

```
FROM python:3.12-slim
RUN useradd --system --no-create-home app
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
USER app
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--access-logfile", "-",
"app:app"]
```

ПРИЛОЖЕНИЕ Б

Docker Compose

```
services:
  db:
    image: postgres:16-alpine
    env_file: .env
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d
$${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 5
    restart: unless-stopped
  app:
    build: .
    image: devops-app:1.0
    env_file: .env
    ports:
      - "127.0.0.1:5000:5000"
    depends_on:
      db:
        condition: service_healthy
    networks:
      - backend
    restart: unless-stopped
volumes:
  db_data:
networks:
  backend:
```

ПРИЛОЖЕНИЕ В

История собранного образа

```
IMAGE            ID             DISK USAGE   CONTENT SIZE   EXTRA
devops-app:1.0   e9fe47b04b0e        211MB           52MB
IMAGE          CREATED          CREATED BY
SIZE      COMMENT
e9fe47b04b0e   6 seconds ago    CMD ["gunicorn" "--bind"
"0.0.0.0:5000" "--a…   0B        buildkit.dockerfile.v0
<missing>      6 seconds ago    EXPOSE [5000/tcp]
0B        buildkit.dockerfile.v0
<missing>      6 seconds ago    USER app
0B        buildkit.dockerfile.v0
<missing>      6 seconds ago    COPY app.py . # buildkit
12.3kB    buildkit.dockerfile.v0
<missing>      7 seconds ago    RUN /bin/sh -c pip install --no-cache-
dir -r…   25.1MB    buildkit.dockerfile.v0
<missing>      15 seconds ago   COPY requirements.txt . # buildkit
12.3kB    buildkit.dockerfile.v0
<missing>      15 seconds ago   WORKDIR /app
8.19kB    buildkit.dockerfile.v0
<missing>      15 seconds ago   RUN /bin/sh -c useradd --system --no-
create-…   41kB      buildkit.dockerfile.v0
<missing>      44 hours ago     CMD ["python3"]
0B        buildkit.dockerfile.v0
<missing>      44 hours ago     RUN /bin/sh -c set -eux;  for src in
idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      44 hours ago     RUN /bin/sh -c set -eux;
savedAptMark="$(a…   41.4MB    buildkit.dockerfile.v0
<missing>      45 hours ago     ENV
PYTHON_SHA256=c2c4321961fab0fb999d66e0ce…   0B
buildkit.dockerfile.v0
<missing>      45 hours ago     ENV PYTHON_VERSION=3.12.15
0B        buildkit.dockerfile.v0
```

```
<missing>      45 hours ago     ENV
GPG_KEY=7169605F62C751356D054A26A821E680…   0B
buildkit.dockerfile.v0
<missing>      45 hours ago     RUN /bin/sh -c set -eux;  apt-get
update;  a…   4.92MB    buildkit.dockerfile.v0
<missing>      45 hours ago     ENV LANG=C.UTF-8
0B        buildkit.dockerfile.v0
<missing>      45 hours ago     ENV
PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B
buildkit.dockerfile.v0
<missing>      2 days ago       # debian.sh --arch 'amd64' out/
'trixie' '@1…   87.7MB    debuerreotype 0.17
```

ПРИЛОЖЕНИЕ Г

Раздел 9 инструкции восстановления

Docker Engine Community 29.9.2 и плагин Compose v5.6.1 установлены из официального
APT-репозитория https://download.docker.com/linux/ubuntu (noble, stable), ключ
/etc/apt/keyrings/docker.asc, источник
/etc/apt/sources.list.d/docker.sources.
Служба docker включена и активна. Пользователь devops входит в группу docker;
новое членство действует в новой SSH-сессии. Доступ к docker.sock предоставляет
административные возможности, поэтому в группу включается только администратор.

Проект находится в /home/devops/app. Доставка с компьютера:

```bash
rsync -avz --delete --exclude=.env --chmod=D755,F644 --dry-run app/ devops:app/
rsync -avz --delete --exclude=.env --chmod=D755,F644 app/ devops:app/
```

Файл /home/devops/app/.env создаётся только на сервере из .env.example,
пароль заменяется результатом `openssl rand -hex 16`, права 600, владелец devops.
Не копировать этот файл на компьютер и не добавлять в Git. .dockerignore исключает
.env из контекста сборки. Команды запуска: `cd ~/app && docker compose up -d --build --wait`.

Образ devops-app:1.0 запускает Gunicorn от пользователя app (uid 999).

PostgreSQL postgres:16-alpine использует именованный том app_db_data и сеть
app_backend. Имя db разрешается встроенным DNS этой сети. Приложение ждёт healthcheck
базы pg_isready. `docker compose down` сохраняет том; `down -v` удаляет данные.
Проверка с удалением выполнена только на созданной учебной базе.

Публикуется только 127.0.0.1:5000 → 5000 приложения. Порт 5432 базы не публикуется.
Compose v5.6.1 в команде `port db 5432` выводил :0 для отсутствующей публикации;
проверка аудита использует реальные NetworkSettings.Ports из docker inspect.

В HTTPS-блок Nginx добавлено:

```nginx
location /app/ {
    proxy_pass http://127.0.0.1:5000/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Проверить синтаксис и применить: `sudo nginx -t && sudo systemctl reload nginx`.
Статический ресурс / сохранён. /app/ показывает счётчик, /app/health отвечает ok.
На Windows используются curl --cacert ~/devops.crt и обычное имя devops.local.

На самой VM в /etc/hosts добавлен devops.local как псевдоним 127.0.0.1 для
локальных диагностических запросов; полное имя узла сохранено.

Проверка состояния: `docker compose ps`, `docker compose exec -T --interactive=false app id`,
`sudo ss -tlnp`, `sudo bash ~/devops-course-2026/scripts/audit.sh`.
Ожидаются healthy у db, только loopback у 5000, отсутствие слушателя 5432,
14 успешных проверок аудита и код 0. При публикации 5000 на всех интерфейсах
аудит возвращает 1; после опыта возвращена безопасная конфигурация.
Снимок 05-docker-compose сохраняет проект, базу и Nginx для следующей работы.